

Ročníkový projekt

Gymnázium, Praha 6, Arabská 14

Programování



Projekt: Převodník Jednotek

Předmět: Programování

Třída: 1.E

Autor: Adam Bárta

Prohlášení

Prohlašuji, že jsem jediným autorem tohoto projektu, všechny citace jsou řádně označené a všechna použitá literatura a další zdroje jsou v práci uvedené. Tímto dle zákona 121/2000 Sb. (tzv. Autorský zákon) ve znění pozdějších předpisů uděluji bezúplatně škole Gymnázium, Praha 6, Arabská 14 oprávnění k výkonu práva na rozmnožování díla (§ 13) a práva na sdělování díla veřejnosti (§ 18) na dobu časově neomezenou a bez omezení územního rozsahu.

V Praze dne 1.května 2026 _____

Anotace

Převodník jednotek je aplikace určená pro převod jednotek a výpočet různých hodnot s jednotkami. Aplikace je vytvořena v programovacím jazyce *Java* a využívá knihovnu *JavaFX* pro zobrazení okenní aplikace. Kromě zadání jedné hodnoty s jednotkou a vypočtení všech možných převodů do dalších jednotek lze v aplikaci provádět i některé fyzikální výpočty. Pokud budou zadány 2 nebo 3 hodnoty, aplikace převede každou z hodnot do základní jednotky a provede výpočet. Např. pokud je zadána hodnota s jednotkou délky a druhá s jednotkou času, aplikace vypočte v metrech za sekundu rychlost a tu vypíše ve všech převoditelných jednotkách.

Abstract

The Unit Converter is an application designed to convert units and calculate various values with units. The application is written in the Java programming language and uses the JavaFX library to display the windowed application. In addition to entering a single value with a unit and calculating all possible conversions to other units, the application can also perform certain calculations. If 2 or 3 values are entered, the application converts each value to the base unit and performs the calculation. For example, if one value is entered with a unit of length and another with a unit of time, the application calculates the speed in meters per second and displays it in all convertible units.

Zadání projektu

Program podle vstupu převede hodnotu dané jednotky do hodnoty požadované jednotky. Při zadání více hodnot určitých jednotek, dokáže spočítat podle vzorce výsledek (například: vypočítá rychlost, objem kváдру, atd.).

1. Úvod

2. Třída Launcher

2.1. Základní nastavení a inicializace

2.2. Tlačítka plus a mínus

2.3. Tlačítko výpočet

2.3.1. Ošetření vstupu

2.3.2. Načtení hodnot

2.3.3. Vyhodnocení výsledků

2.3.4. Výpis výsledků

2.4. Zobrazení scény

3. Třída Hodnoty

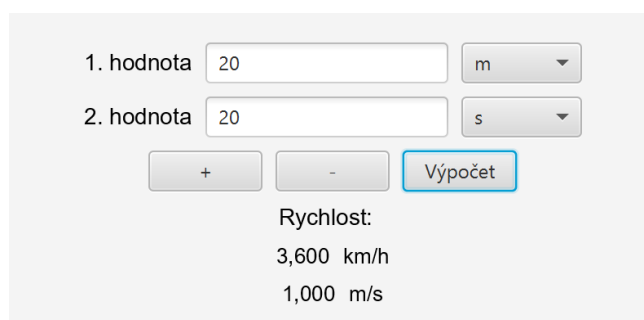
3.1. Metoda ZapisDoPole

4. Závěr

5. Zdroje

1. Úvod

Převody jednotek jsou důležité jak při profesionálních technických výpočtech, tak i v každodenním životě. Převodník jednotek tento proces zjednodušuje a rovnou některé časté fyzikální výpočty automaticky vypočítává. V této dokumentaci najdete jak celá aplikace funguje, včetně algoritmu pro převádění a výpočet hodnot a nastavení uživatelského rozhraní.



Výpočet rychlosti a převedení na jednotky

2. Třída Launcher

Třída Launcher se spouští společně se spuštěním programu. Nastavuje základní parametry okna, zobrazuje všechny prvky, získává přes okno vstup od uživatele a zobrazuje výstup. Celé zobrazení je tvořeno pomocí rozvržení typu *VBox* (naskládá prvky pod sebe) do kterého jsou naskládány jednotlivé *HBoxy* (skládá prvky vedle sebe – fungují jako řádky).

2.1. Základní nastavení a inicializace

Program začíná vytvořením atributů *pocet_poli*, *povolitpokracovani* a *pocet_hboxu*. Atribut *pocet_poli* počítá, kolik je právě zobrazeno textových polí. Při spuštění je přidáno na obrazovku pouze jedno pole, proto je při spuštění programu nastavena tato hodnota na 1. Atribut *povolitpokracovani* slouží k zamezení výpočtu, pokud uživatel nezadal všechny potřebné hodnoty. Více o tomto atributu se tak můžete dozvědět v kapitole **Ošetření vstupu**. Dále program vytvoří atribut *pocet_hboxu*. Ten počítá, na který *HBox* (každý řádek je vypsán pomocí *HBoxu*) bude potřeba vložit další text nebo jednotku. Tento atribut je použit především při výpisu hodnot a je tak o něm více v kapitole **Výpis výsledků**.

Na začátku metody *start* je přidán hlavní *VBox* (v programu jako *svisly_panel*) a pole *HBoxů* (v programu jako *hbox[]*). Bylo vybráno pole, protože je jich potřeba hodně (při výpisu je potřeba na každý převod jeden řádek) a lépe se s nimi ve větším množství

manipuluje. Objekty typu *Text*, *TextField* a *ChoiceBox* jsou také pole. Index polí instancí těchto objektů se vždy shoduje s indexem *HBoxu*, ve kterém se nachází. Dále jsou nastavena tlačítka (objekty typu *Button*) +, - a výpočet spolu s jejich parametry (v parametrech jejich vizáží).

Pomocí prvního cyklu jsou vytvořeny *HBoxy* a instance objektů, které slouží k zadání vstupu. *HBoxy* jsou rovnou vloženy do hlavního *VBoxu*. Všechny objekty jsou zatím uloženy pouze v paměti, protože se budou postupně přidávat, až je tlačítkem uživatel přidá.

Ze začátku jsou tak přidány na obrazovku do prvního *HBoxu* jen první objekty, které se zobrazí jako první - *tx[1]*, *txF[1]* a *cB[1]*. Do druhého pak tlačítka, pomocí kterých může uživatel přidat další a tím tak prostor pro další hodnoty. Více jsou tlačítka popsána v kapitole **Tlačítko plus a mínus**.

Pro vytvoření polí instancí objektů, které slouží pouze k vypisování výstupu slouží druhý cyklus. Abych jednoduše rozlišil hodnotu od jednotky, hodnota je vypisována objektem s názvem *tx* a jednotky *tx2*. Tato pole mají index od 4 (aby se shodovala s indexem *HBoxu*, objekty *tx* s indexem 1-3 slouží jako orientační prvek při zadávání vstupu, *tx[3]* lze někdy použít i pro výpis – více v kapitole **Tlačítko plus a mínus**). Oproti prvnímu cyklu, jsou do *HBoxů* přidány jednotlivé instance. Všechny řádky jsou znovu vloženy do *VBoxu*.

```
25 VBox svisly_panel = new VBox(); // VYTVOŘENÍ BORDERU A VBoxu
26
27 HBox[] hbox = new HBox[25]; // VYTVOŘENÍ 25 HBoxů
28
29 Text[] tx = new Text[25]; // 25 TEXTOVÝCH POLÍ
30 Text[] tx2 = new Text[25]; // 25 TEXTOVÝCH POLÍ PRO VYPISÁNÍ JEDNOTEK
31 TextField[] txF = new TextField[4]; // 4x TEXTOVÉ POLE PRO ZADÁNÍ HODNOT
32 ComboBox[] cB = new ComboBox[4]; // 4x COMBOBOX PRO ZADÁNÍ JEDNOTEK
33
34 Button bt1 = new Button(s: "+");
35 Button bt3 = new Button(s: "-");
36 Button bt2 = new Button(s: "Výpočet"); // VYTVOŘENÍ BUTTONŮ
37
38 bt1.setPrefWidth(70);
39 bt3.setPrefWidth(70);
40 bt2.setPrefWidth(70); // VYTVOŘENÍ BUTTONŮ
41
42 svisly_panel.setSpacing(8); // NASTAVENÍ ODSAZENÍ MEZI HBoxY
43 svisly_panel.setPadding(new Insets(w: 20));
```

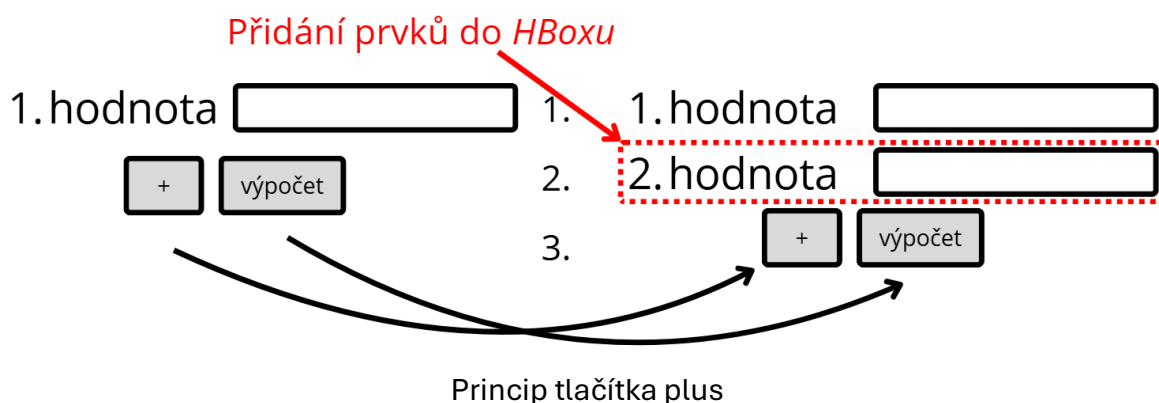
Nastavení základních parametrů

2.2. Tlačítko plus a mínus

Tlačítko plus (v programu jako objekt *bt1*) slouží k rozšíření nabídky textových polí, které umožňují přidat více hodnot a provést výpočty. Funguje tak, že podle počtu polí vždy přesune tlačítka o úroveň níže a přidá na uvolněný prostor další textové pole. Protože je 3. *HBox* někdy využíván pro výpis výsledků (pokud počet polí je jedna) a někdy

pro vstup, je preventivně textové pole ze 3. *HBoxu* odstraněno, aby nedošlo ke dvojitému přidání textového pole. Dále pomocí metody *vymazatMtd* nastaví všechny instance objektů sloužících k výpisu na prázdné (text “”), protože se přidáním dalšího textového pole výpis rozbije.

Tlačítko mínus (v programu označováno jako objekt *bt3*) funguje velice podobně jako *bt1*, avšak s rozdílem, že výsledek po stisknutí tlačítka je odebrání textového pole.



2.3. Tlačítko výpočet

Po stisknutí tlačítka výpočet (v programu jako *bt2*), dojde ke shromáždění dat a jejich předání do třídy *Hodnoty*, která data zpracuje a vrátí hodnoty a jednotky v uceleném pořadí. Na základě hodnot a jednotek provede výpočty nových veličin a vypíše převedené jednotky. Více o tom, jak funguje tento algoritmus, najdete v kapitole **Metoda ZapisDoPole**.

2.3.1. Ošetření vstupu

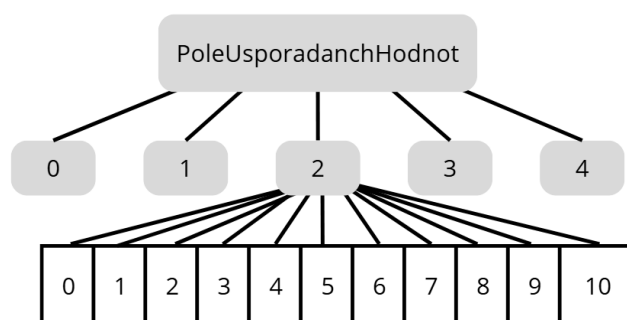
Pokud by uživatel nezadal kompletní data, nebo nezadal čísla, program by je nemohl zpracovat. Proto jsem přidal atribut *povolitpokracovani*. Ten je ze začátku nastaven na nulu. Nula přístup povoluje a 1 zakazuje.

Nejprve dojde k vytvoření pole jednotek a hodnot (*pole_jednotek* a *pole_hodnot*). Do těchto program uloží všechny zadané hodnoty. Protože výstup z textového pole nevychází v datovém typu *double*, ve kterém algoritmus počítá, bylo potřeba ho pro pole hodnot do *double* převést pomocí metody *Double.valueOf()*. Rozběhne se cyklus, který pomocí výjimky nahrává do polí hodnoty a jednotky a v případě chybějící informace, nebo zadání jiných znaků než číslíc, zastaví cyklus. Pokud se tak stane nastaví objekt *tx[1]* (standardně využíván pro zobrazení “1. hodnota”) na “Neplatný zápis!”, pro informování uživatele. Tento text je vždy při stisku tlačítka nebo při úspěšném nahrání hodnot vyresetován zpět, aby se nezobrazovalo upozornění, pokud už byly hodnoty opraveny. Dále nastaví atribut *povolitpokracovani* na 1, čímž znemožní

provedení výpočtu a přeskočí a ukončí cyklus. Po přeskočení hlavní části s výpočtem nastaví *povolitpokracovani* opět na nulu, a udělí šanci uživateli zkoušet hodnoty zadávat znovu.

2.3.2. Načtení hodnot

Pokud nebyl cyklus zastaven kvůli chybějícím datům, program má načtené *pole_hodnot* a *pole_jednotek* a pokračuje dál. Dojde k vytvoření tří polí, která budou uchovávat informace. *PoleUsporadanchHodnot* uchovává všechny již vypočítané a ucelené hodnoty a *PoleUsporadanychJednotek* na jejich jednotky. První pole *PoleUsporadanchHodnot* (v první hranaté závorce) označuje pole již převedených hodnot kolem první zadané hodnoty na první textovém poli. Druhé a třetí pole funguje stejně, jen pro další textová pole. Čtvrté pole je pole vypočtených hodnot objemu při zadání 3 hodnot. *PoleUsporadanychJednotek* má na stejných indexech jako *PoleUsporadanchHodnot* přiřazené jednotky. Nulté pole u *PoleUsporadanychJednotek* funguje jako komunikační kanál. Výpočet hodnot do těchto polí vyplní poté pomocí cyklu třída *Hodnoty* a její metoda *ZapisDoPole*. Více o této třídě a jejích metodách najdete v **kapitole Třída Hodnoty**.



PoleUsporadanchHodnot

2.3.3. Vyhodnocení výsledků

Jakmile proběhne výpočet pomocí třídy *Hodnoty*, je nutné výsledky před výpisem vyhodnotit. Je potřeba k nim doplnit titulek a pokud je to možné, vypočítat ostatní hodnoty.

Pokud *pocet_poli* je jedna (byla zadána jen jedna hodnota), znamená to, že je potřeba vypsát výsledek a vypočítat obvod a obsah kruhu. Pro jakékoliv dodatečné výpočty jsem vytvořil metodu *NovyVyras*, protože bude tento proces prováděn vícekrát. Do ní se jen vloží požadovaný výpočet, jednotku výpočtu a index *PoleUsporadanchHodnot*, na který bude pole ucelených a převedených hodnot zadané hodnoty uloženo. Další pole hodnot je možné uložit na jakýkoliv index kromě 1, takže

jsem vybral na obvod kruhu 2 a obsah kruhu 3, které jsou prázdné. Na nultou pozici *PoleUsporadanychJednotek* je doplněn titulek. Tento titulek bude zobrazen a program podle něho dokáže identifikovat, zda má dané pole zobrazit.

Jestliže je *pocet_poli* 2, vždy se vypočte podle zadaných jednotek nová hodnota a jen ta bude vypočítána a zobrazena. U počtu polí 2 jsem znovu všude použil metodu *NovyVyzraz* pro vypočtení nových hodnot. Pokud se z jednotek nebude moct vytvořit žádná nová hodnota, program nastaví na *PoleUsporadanychJednotek[0][0]* “nedostupne prevody” a program tak později nic nevypíše. Tato hodnota je vždy na začátku výpočtu resetována na “volno”, takže lze poté zkoušet zadat jiné hodnoty.

Pokud je *pocet_poli* 3, výpočet probíhá stejně jako u dvou. Lze zde ale vypočítat pouze objem, a to pouze, pokud uživatel zadá jenom jednotky délky.

2.3.4. Výpis výsledků

Pokud tedy nejsou žádné problémy s výpočty (*PoleUsporadanychJednotek[0][0]* je “volno”), program může pokračovat, jinak je nastaven *tx[1]* na “Neplatný zápis!”. Pokud je *pocet_poli* 1, je potřeba vypsát na třetí řádek první popisek (na druhém jsou tlačítka, proto až 3.). Proto pokud zde na není 3. *HBoxu* umístěn (pokud *pocet_poli* je 1, textové pole není na 3. *HBoxu* přidán), umístil jsem ho. Následně se už zahajuje cyklus, který vypisuje popisky a spouští hlavní cyklus. Každé opakování postupuje, ale pouze tehdy, pokud na komunikačním kanálu (*PoleUsporadanychJednotek[0]*) se něco vyskytuje. Pokud ano, popisek bude vypsán a spustí se vnořený hlavní cyklus.

Pak se spustí vnořený cyklus, ve kterém je vypsáno celé pole ucelených hodnot a jednotek. Znovu jsem přidal ověření, jestli pole není prázdné, jinak se vypíše hodnota a jednotka na daném *HBoxu*. Hodnota je datového typu *double* a zároveň není dostatečně zaokrouhlená. Tento problém jsem vyřešil pomocí *String.format("%.3f", PoleUsporadanychHodnot[b][d])*. Tato metoda zaokrouhlí číslo na tři desetinná místa a převede ho do datového typu *string*, abych ho mohl vypsát jako text.

2.4. Zobrazení scény

Na konci programu je přidán *VBox* do scény a jsou nastaveny počáteční rozměry okna. Vybral jsem 400x560, protože aplikace zobrazí všechny výsledky přehledně i při výpisu hodnot s jednou vstupní hodnotou a zároveň není zbytečně velká. Na konci dojde k zobrazení scény.

3. Třída Hodnoty

Třída *Hodnoty* zpracovává zadanou hodnotu a jednotku. Výsledkem je převedení zadané hodnoty do více převodů, a to doplněním do *PoleUsporadanychHodnot* a

PoleUsporadanychJednotek. Pro rychlejší a jednodušší vytvoření nové instance jsem vytvořil konstruktor, který doplní atribut *Hodnota* a *Jednotka*.

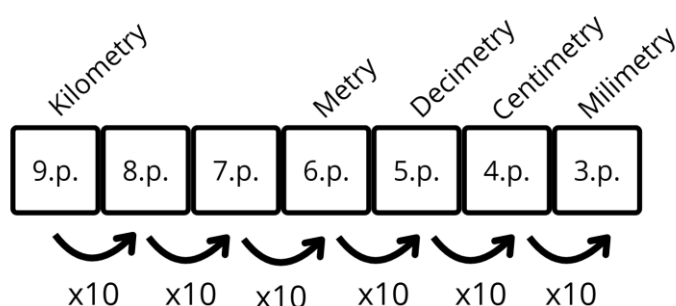
3.1. Metoda ZapisDoPole

Třída *Hodnoty* vždy pracuje s vedlejší třídou pomocí metody *ZapisDoPole*. Tato metoda ovládá všechny ostatní metody v této třídě. Nejdříve je instanci třídy přiděleno pořadí pomocí metody *PriradPoradi()*. Pořadí přiděluje hodnotě váhu a slouží k určení vzdálenosti mezi jednotlivými jednotkami, mezi kterými je hodnota převáděna. Pořadí se shoduje s indexem, na který je hodnota uložena. Díky pořadí je možno přiřadit zadanou hodnotu a jednotku na správný index *PoleUsporadanychHodnot*. Díky pořadí je tak můžeme umístit na správné místo. Poté je pomocí metody *PriradSkupinu()* přidělena skupina. Podle skupiny probíhá výpočet převodů.

V této chvíli má program všechna potřebná data a může doplňovat do pole ostatní hodnoty. Každá jednotka se převádí jinak, proto jsem rozdělil různé násobení podle skupiny. Program vždy postupuje pomocí cyklu a jednotlivě do pole doplňuje.

Doplňuje tak, že vezme hodnotu, kterou zadal uživatel. Tu vynásobí číslem, kterým se posouvá desetinná čárka a s exponentem rozdílu pořadí (to znázorní vzdálenost mezi hodnotami).

Pro příklad, program bude doplňovat jednotky délky. Bude vyplňovat jednotky v mm a uživatel zadal hodnotu 20 v jednotkách m. 10 umocníme na $6-3$ (metry mají pořadí 6, milimetry pořadí 3), tedy na 3. Toto číslo bude vynásobeno 20 a výsledek převodu 20 m je tak 20 000 mm. Pořadí 7 a 8, protože na nich žádná jednotka není, jen utváří vzdálenost mezi metry a kilometry. Tato situace je vyobrazena na obrázku.



Demonstrace převodu jednotek délky (p. = pořadí)

Pro ostatní jednotky program funguje podobně, jen s malými modifikacemi odpovídající možným jednotkám.

Největším problémem bylo pro mě vytvořit skupinu času, která se na některých místech násobí násobkem úplně jiným. Mezi sekundou, minutou a hodinou se násobí

standardně 60, ale mezi hodinou a dnem 24. Proto jsem program v cyklu rozdělil na 2 větve. Jedna je, pokud je jednotka zadané hodnoty pořadí 9 (den) a druhá, pokud není. Pokud je, program násobí standardně, až na to, že každý výpočet násobí 24 (aby se dostal na hodiny) a rozdíl pořadí je zmenšen o 4, protože rozdíl pořadí mezi hodinou a dnem je nahrazen vynásobením. Pokud zadaná hodnota není jednotky den (pořadí 9), znovu je program rozdělen. Když ani doplňovaná jednotka není den (na pořadí 9), násobí zcela normálně s realistickým rozdílem pořadí. Pokud dopočítává a vyplňuje index 9 (den), jednoduše vydělí hodnotu indexu 5 (hodina) dvaceti čtyřmi.

4. Závěr

Zadání projektu se mi podařilo úspěšně splnit a zároveň jsem navíc od zadání vytvořil okenní aplikaci pro snadnější interakci s uživatelem. Podařilo se mi vyřešit problémy týkající se různých převodů jednotek. Prací na projektu jsem si prohloubil schopnosti pracovat s třídami a objekty. Naučil jsem se alespoň základně pracovat s knihovnou *JavaFX* a vytvářet pomocí ní okenní aplikace. V budoucnu by mohla být dále rozvíjena a to např. rozšířením databáze o další fyzikální výpočty.

5. Zdroje

- Google, model: Gemini 1.5 Flash, dostupné z: <https://gemini.google.com>

Při vytvoření okenní aplikace v *JavaFX*, byla použita umělá inteligence Gemini, k získání informací ohledně různých objektů. Dotazy byly pokládány většinou ve formátu např.: „Jaký objekt vytvoří rolovací nabídku v *JavaFX*?” nebo „Jak nastavím odsazení od okrajů objektů v *JavaFX*?” Umělá inteligence Gemini byla použita také pro konzultaci ohledně různých implementací metod a podobně, např.: „Jak mohu převést datový typ *Double* na *String* v *JavaFX*?” Většinou byly pokládány i dotazy rozšiřující.